

APPLIED AI ENGINEERING SERIES

Retrieval-Augmented Generation (RAG)

A Complete Basic-to-Advanced Handbook — Pipelines, Architectures, Code,
Evaluation & Production Engineering

Indexing

Retrieval

Re-ranking

GraphRAG

Evaluation

Production

From naive vector search to agentic, self-correcting, production-grade retrieval systems

Table of Contents

Part A — RAG Fundamentals

- What RAG is and why it exists
- RAG vs Fine-tuning vs Long-context vs Prompt engineering
- The core RAG pipeline end-to-end
- When RAG is (and isn't) the right choice

Part B — The Indexing Pipeline

- Document loading & parsing
- Chunking strategies (fixed, recursive, semantic, structural)
- Embeddings explained
- Vector databases & index types (HNSW, IVF, Flat)
- Metadata design

Part C — The Retrieval Pipeline

- Dense vs sparse retrieval (BM25, vectors)
- Hybrid search & fusion (RRF)
- Re-ranking with cross-encoders
- Query transformation (rewriting, HyDE, multi-query, decomposition)
- Metadata filtering & self-query

Part D — Generation & Prompting

- Context assembly & prompt design
- Grounding, citations & refusal behavior
- Context window management & the 'lost in the middle' problem

Part E — Advanced RAG Architectures

- Naive → Advanced → Modular RAG
- Parent-document & small-to-big retrieval
- Contextual retrieval
- GraphRAG
- Agentic / Self-RAG & CRAG
- Multimodal RAG

Part F — Evaluation & Observability

- Retrieval metrics (Recall@k, MRR, nDCG)
- Generation metrics (faithfulness, relevance)
- The RAGAS framework
- Building an eval set & CI for RAG

Part G — Production Engineering

Cost, latency & caching
Chunk/index refresh & incremental updates
Security: access control, PII, prompt injection
Common failure modes & debugging playbook

Part H — Reference Implementations & Interview Prep

End-to-end code: minimal RAG
End-to-end code: advanced hybrid + rerank RAG
Tech stack comparison tables
Common RAG interview questions & answers

PART A

RAG Fundamentals

What Is RAG and Why Does It Exist?

Retrieval-Augmented Generation (RAG) is an architecture that connects a language model to an external knowledge source at inference time. Instead of relying only on what the model memorized during training, the system *retrieves* relevant documents for the user's question and places them into the prompt so the model can *generate* an answer grounded in that retrieved evidence.

The four problems RAG solves:

- **Knowledge cutoff** — models don't know anything after their training data ends. Retrieval injects fresh information.
- **Private/proprietary data** — your internal wiki, contracts, tickets, and codebase were never in the training set.
- **Hallucination** — given grounded source text, a model is far less likely to invent facts, and you can require citations.
- **Traceability** — RAG answers can point to source documents, which is mandatory in legal, medical, and financial settings.

The Mental Model

Think of a closed-book exam versus an open-book exam. A bare LLM is a student answering from memory — fluent, but liable to misremember. RAG turns it into an open-book exam: the student still has to reason and write the answer, but now has the textbook open to the right page. **The quality of a RAG system is therefore mostly determined by whether you turned to the right page** — which is why most of this handbook is about retrieval, not generation.

RAG vs Fine-tuning vs Long-Context vs Prompting

Approach	Best For	Limitations	Cost Profile
RAG	Injecting factual, changing, or private knowledge; anything needing citations	Adds retrieval latency; quality capped by retriever; needs an indexing pipeline	Low upfront, moderate per-query (embedding + retrieval + larger prompt)

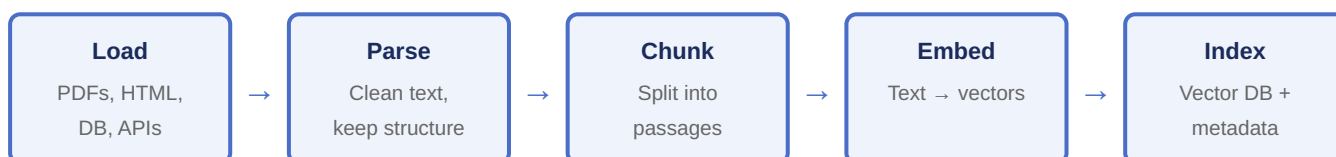
Fine-tuning	Teaching <i>behavior</i> , style, output format, domain tone, or a narrow skill	Poor for facts (they go stale); needs labeled data; retraining to update	High upfront training cost, cheap inference
Long-context stuffing	Small, fixed corpora that fit in the window (e.g., one contract, one codebase module)	Expensive per call; degrades with distraction; doesn't scale to millions of docs	Very high per-query token cost
Prompt engineering	Shaping reasoning/format on knowledge the model already has	Cannot add new knowledge	Cheapest

Interview-ready framing: "RAG teaches the model *what it knows*; fine-tuning teaches it *how to behave*." In practice mature systems combine both — fine-tune for format/tone/domain vocabulary, and use RAG for facts. Long context does not kill RAG: retrieval is also a cost-control and precision mechanism, not just a context-window workaround.

The Core RAG Pipeline End-to-End

Every RAG system, no matter how advanced, is two pipelines: one that runs **offline** (indexing) and one that runs **online** (query time).

OFFLINE — INDEXING PIPELINE (RUNS ON INGEST/REFRESH)



ONLINE — QUERY PIPELINE (RUNS PER USER REQUEST)



Where quality is actually won or lost (from most to least impactful):

1. **Chunking & parsing quality** — garbage chunks cannot be rescued by any downstream step.
2. **Retrieval recall** — if the right chunk is never retrieved, the answer cannot be correct.
3. **Re-ranking precision** — getting the right chunk into the top 3–5 rather than merely the top 50.
4. **Prompt/grounding design** — instructing the model to answer only from context and to say "I don't know."
5. **Model choice** — usually the smallest lever, and the one teams wrongly optimize first.

When RAG Is (and Isn't) the Right Choice

Use RAG when...	Don't use RAG when...
Knowledge changes frequently or is private	The task is pure reasoning/math with no external facts
You must cite sources or show provenance	The whole corpus comfortably fits in the context window and is static
The corpus is large (thousands to billions of tokens)	You need to change the model's <i>style</i> or output format — fine-tune instead
Answers must be auditable/verifiable	Latency budget is extremely tight (<200ms) and answers are generic

PART B

The Indexing Pipeline

Step 1 — Document Loading & Parsing

This unglamorous step causes more RAG failures than any modeling decision. A PDF table flattened into scrambled text, or a page header repeated into every chunk, silently poisons retrieval forever.

Source Type	Parsing Challenge	Practical Approach
Native/text PDFs	Multi-column layout, headers/ footers, footnotes	Layout-aware parsers; strip repeating headers; preserve reading order
Scanned PDFs / images	No text layer at all	OCR first; consider vision-model extraction for complex layouts
Tables	Row/column relationships destroyed when flattened	Extract to Markdown/HTML tables; optionally LLM-summarize each table into prose for embedding
HTML / web	Nav bars, ads, boilerplate dominate the text	Main-content extraction; keep heading hierarchy as metadata
Code repos	Splitting mid-function destroys meaning	AST/syntax-aware splitting by function/class
Slides (PPTX)	Text fragments without narrative flow	Merge slide title + body + speaker notes into one unit

Classic failure: A 10-K financial PDF parsed naively yields chunks like "...2,481 1,932 4,113 (22)%..." with no column headers. Retrieval may surface it, but the model cannot interpret it. **Fix:** extract tables structurally and attach a natural-language caption before embedding.

Step 2 — Chunking Strategies

Chunking decides the atomic unit of retrieval. Too small and chunks lack context; too large and embeddings become diluted averages that match nothing precisely.

Strategy	How It Works	Pros / Cons	Use When
Fixed-size	Split every N tokens/chars, usually with overlap	+ Trivial, predictable – Cuts mid-sentence, ignores structure	Baselines, homogeneous prose

Recursive character	Try splitting on paragraphs, then sentences, then words, until under size limit	+ Respects natural boundaries – Still structure-blind	Sensible default for most text
Document-structural	Split on Markdown headings, HTML sections, PDF sections	+ Semantically coherent, great metadata – Requires clean structure	Docs, wikis, manuals, policies
Semantic	Embed sentences; cut where consecutive-sentence similarity drops sharply	+ Topic-aligned chunks – Slow & costly to index	Unstructured narrative text
Proposition / atomic	LLM rewrites text into standalone factual statements	+ Very high precision – Expensive, can lose nuance	Fact-heavy QA, compliance
Code-aware (AST)	Split by function/class boundaries	+ Keeps units executable/meaningful – Language-specific	Code assistants

Practical chunking heuristics:

- Start at **400–800 tokens** per chunk with **10–15% overlap**; tune with an eval set rather than by intuition.
- Overlap exists to stop a fact being severed at a boundary — it is a cheap insurance policy, not a quality strategy.
- **Always prepend context to the chunk text before embedding**: document title, section heading, and date. A chunk reading "It must be filed within 30 days" is useless; "Form 12B / Tax Filing Policy 2024 — It must be filed within 30 days" is retrievable.
- Store the **parent document ID and character offsets** so you can expand a matched chunk back to its surroundings at generation time.
- Chunk size interacts with your embedding model's max sequence length — exceeding it silently truncates.

```
# Structural chunking with context injection (the single highest-ROI chunking trick)
def make_chunks(doc):
    chunks = []
    for section in split_by_headings(doc.text):          # structural split
        for piece in recursive_split(section.body, size=600, overlap=80):
            # Embed the piece WITH its breadcrumb context
            embed_text = f"{doc.title} > {section.heading}\n\n{piece}"
            chunks.append({
                "embed_text": embed_text,                # what gets vectorized
                "raw_text": piece,                        # what gets shown to the LLM
                "metadata": {
                    "doc_id": doc.id, "title": doc.title,
                    "section": section.heading, "date": doc.updated_at,
                    "source_url": doc.url, "access_level": doc.acl,
                }
            })
    return chunks
```

Step 3 — Embeddings Explained

An embedding model maps text to a dense vector (typically 384–3072 dimensions) such that semantically similar texts land close together. Retrieval then becomes a nearest-neighbor search in that space.

Key concepts you must be able to explain:

- **Cosine similarity** — the standard distance metric; measures angle, ignoring magnitude. Range -1 to 1 .
- **Bi-encoder** — encodes query and document *separately*, so document vectors can be precomputed and indexed. Fast, used for retrieval.
- **Cross-encoder** — encodes query and document *together* for a much more accurate relevance score, but cannot be precomputed. Slow, used for re-ranking a shortlist.
- **Asymmetric search** — queries are short, documents long. Some models expect prefixes like `query: / passage:` ; omitting them measurably degrades quality.
- **Matryoshka embeddings** — trained so a truncated prefix of the vector still works, letting you trade dimensions for storage/speed.

Choosing an embedding model — what matters	Why
Retrieval benchmark performance (e.g., MTEB) on your domain	Leaderboard rank on generic data often fails to transfer to legal/medical/code corpora
Max sequence length	Must exceed your chunk size or text is silently truncated
Dimensionality	Directly drives index size, RAM, and query latency

Multilingual support	Essential if queries and documents differ in language
Self-hosted vs API	Data residency, cost at scale, and latency control

Critical operational rule: The query and the documents must be embedded with the **same model and version**. Changing embedding models requires a **full re-index of the entire corpus** — budget for it, and version your index accordingly.

Step 4 — Vector Databases & Index Types

Exact nearest-neighbor search is $O(N)$ per query and doesn't scale, so vector DBs use **Approximate Nearest Neighbor (ANN)** indexes that trade a little recall for orders-of-magnitude speed.

Index	Mechanism	Trade-off	When to Use
Flat (brute force)	Compare against every vector	100% recall, slow at scale	<100k vectors, or ground-truth evaluation
HNSW	Multi-layer navigable small-world graph; greedy descent	Excellent speed/recall, high memory	Default choice for most production systems
IVF	Cluster vectors; search only the nearest nprobe clusters	Lower memory, needs training step, recall tuned via nprobe	Very large corpora, memory-constrained
PQ / IVF-PQ	Compress vectors into quantized codes	Massive memory savings, some accuracy loss	Billion-scale corpora

HNSW parameters you should know by name:

- **M** — edges per node. Higher = better recall, more memory.
- **ef_construction** — search breadth while building. Higher = better index, slower build.
- **ef_search** — search breadth at query time. **The main runtime recall/latency dial**; tune this per query load.

Vector Store	Character	Good Fit
FAISS	Library, not a server; extremely fast, no persistence/filters built in	Research, embedded, offline eval
pgvector (Postgres)	Vectors inside your existing relational DB, real SQL filtering & joins	Teams already on Postgres; strong default
Managed vector DBs	Hosted, scalable, hybrid search & metadata filters built in	Fast production ramp without ops burden
Elasticsearch / OpenSearch	Mature BM25 + vector in one engine	Hybrid search where keyword matters a lot

Underrated advice: If your corpus is under a few hundred thousand chunks and you already run Postgres, `pgvector` with a good hybrid query is usually the right answer. Reach for a dedicated vector database when scale, filtering complexity, or ops requirements genuinely demand it.

Step 5 — Metadata Design

Metadata is what turns a toy demo into a usable system: it enables filtering, permissions, recency weighting, and citation rendering. Design it deliberately at ingest — retrofitting is painful.

Metadata fields worth storing on every chunk:

- `doc_id` , `chunk_id` , `chunk_index` — identity and ordering for parent expansion
- `title` , `section_path` — for context injection and citation display
- `source_url` / `file_path` , `page_number` — so users can verify the claim
- `created_at` , `updated_at` , `version` — for recency filters and stale-document cleanup
- `access_level` / `tenant_id` / `allowed_groups` — **mandatory** for multi-tenant or enterprise deployments
- `doc_type` , `language` , `author` — routing and filtering dimensions

Security note: Access control must be enforced as a **pre-filter inside the retrieval query**, never by filtering results after generation. If an unauthorized chunk reaches the prompt, it has already leaked — the model may paraphrase it even if you drop the citation.

PART C

The Retrieval Pipeline

Retrieval is where RAG systems are made or broken. The generation model can only work with what you hand it — so every technique below exists to raise the odds that the correct passage is in the top few results.

Dense vs Sparse Retrieval

	Sparse (BM25 / keyword)	Dense (embeddings)
Matches on	Exact lexical overlap, term frequency	Semantic meaning in vector space
Strength	Rare terms, IDs, error codes, product names, acronyms, names	Paraphrase, synonyms, conceptual similarity
Weakness	Vocabulary mismatch ("car" vs "automobile")	Misses exact identifiers; "ERR-5012" may embed close to unrelated codes
Cost	Cheap, no model needed	Embedding inference + vector index
Cold start	Works immediately	Needs embedding of the whole corpus

The practical conclusion: These failure modes are complementary, not competing. Almost every strong production RAG system runs **both** and fuses the results — this is the single highest-ROI retrieval upgrade over naive vector-only search.

Hybrid Search & Reciprocal Rank Fusion (RRF)

Hybrid search runs sparse and dense retrieval in parallel and merges the ranked lists. Scores from BM25 and cosine similarity are not on a comparable scale, so the robust merge method is **rank-based**, not score-based.

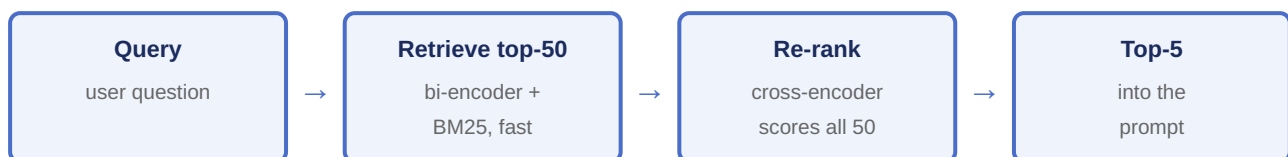
```
# Reciprocal Rank Fusion: score by POSITION, not by raw score.
# k=60 is the standard damping constant from the original paper.
def rrf_fuse(ranked_lists, k=60, weights=None):
    scores = {}
    weights = weights or [1.0] * len(ranked_lists)
    for lst, w in zip(ranked_lists, weights):
        for rank, doc_id in enumerate(lst, start=1):
            scores[doc_id] = scores.get(doc_id, 0) + w * (1.0 / (k + rank))
    return sorted(scores, key=scores.get, reverse=True)

bm25_hits = bm25_search(query, top_k=50)      # lexical
vector_hits = vector_search(embed(query), top_k=50) # semantic
fused = rrf_fuse([bm25_hits, vector_hits], weights=[0.4, 0.6])[:20]
```

Why RRF over score normalization: RRF needs no tuning per-corpus, is immune to score-scale drift when you swap embedding models, and degrades gracefully if one retriever returns junk. It's the default for good reason.

Re-ranking with Cross-Encoders

Retrieval optimizes for **recall** (get the right doc somewhere in the top 50). Re-ranking optimizes for **precision** (get it into the top 3). This two-stage design is the backbone of modern search.



Why re-ranking works so well:

- A bi-encoder compresses a document into one vector *before ever seeing the query* — information is necessarily lost.
- A cross-encoder processes **[query, document]** jointly with full attention, so it can judge subtle relevance (e.g., the doc mentions the right term but in the wrong sense).
- Cost is bounded because you only score a shortlist (50–100), not the whole corpus.
- Typical measured gain: **+10–25% on nDCG@5** over retrieval alone — usually the second-biggest win after hybrid search.

```
candidates = hybrid_retrieve(query, top_k=50)

# Cross-encoder scores each (query, doc) pair jointly – slow but accurate
pairs = [(query, c.text) for c in candidates]
scores = cross_encoder.predict(pairs)

reranked = [c for _, c in sorted(zip(scores, candidates),
                                key=lambda x: x[0], reverse=True)]
context = reranked[:5] # only these go into the prompt
```

Latency reality: Cross-encoder re-ranking adds roughly 50–300ms for 50 candidates. If that breaks your budget, shrink the candidate set to 20, use a smaller re-ranker, or re-rank only when the top retrieval scores are close together (an adaptive strategy).

Query Transformation

Users write short, ambiguous, context-dependent queries. Documents are written in formal, complete prose. Query transformation closes that gap *before* retrieval runs.

Technique	What It Does	Example	Best For
Query rewriting	LLM rewrites a vague/conversational query into a standalone, searchable one	"what about the second one?" → "What are the pricing terms of the Enterprise plan?"	Multi-turn chat (essential)
Multi-query expansion	Generate N paraphrases, retrieve for each, fuse results	One question → 3 phrasings → RRF merge	Boosting recall on ambiguous queries
HyDE	LLM writes a <i>hypothetical answer</i> , then embeds that instead of the question	Question-shaped text is embedded as answer-shaped text	Fixing question/passage asymmetry
Decomposition	Split a multi-part question into sub-questions, retrieve for each	"Compare A and B on cost and security" → 4 sub-queries	Comparative & multi-hop questions
Step-back prompting	Ask a more general question first to retrieve background, then the specific one	"Why did X fail in Q3?" → also "What is X's business model?"	Reasoning-heavy questions
Self-query	LLM extracts metadata filters from natural language	"2023 security incidents" → filter <code>year=2023, type=security</code>	Corpora with rich metadata

Conversational RAG trap: Never embed the raw follow-up message. "And the second one?" retrieves nothing useful. **Always run history-aware query rewriting first** — this is the #1 bug in chat-based RAG systems.

```
# History-aware rewriting – mandatory for any multi-turn RAG chatbot
REWRITE_PROMPT = """Given the conversation history and the latest user message,
rewrite the latest message as a standalone search query that makes sense
without the history. Output ONLY the rewritten query.
```

```
History:
{history}
```

```
Latest message: {question}"""
```

```
standalone_q = llm(REWRITE_PROMPT.format(history=hist, question=q))
docs = hybrid_retrieve(standalone_q)      # retrieve with the REWRITTEN query
answer = llm(build_prompt(docs, q))       # generate with the ORIGINAL query
```

PART D

Generation & Prompting

Context Assembly & Prompt Design

Once you have the top-k chunks, how you present them materially affects faithfulness, citation accuracy, and refusal behavior.

```
SYSTEM = """You are a precise assistant that answers ONLY from the provided context.
```

```
Rules:
```

- Use only information in the CONTEXT block. Do not use prior knowledge.
- Cite the source id in square brackets after each claim, e.g. [doc_3].
- If the context does not contain the answer, reply exactly:
"I don't have enough information in the provided documents to answer that."
- Do not speculate, extrapolate, or fill gaps with plausible-sounding detail.
- If sources conflict, state the conflict and cite both."""

```
# Number sources explicitly so citations are unambiguous
```

```
context = "\n\n".join(  
    f"[doc_{i}] (source: {d.title}, {d.date})\n{d.text}"  
    for i, d in enumerate(reranked[:5], 1)  
)
```

```
user_msg = f"CONTEXT:\n{context}\n\nQUESTION: {question}"
```

Prompt design rules that measurably reduce hallucination:

- **Give an explicit escape hatch.** Models hallucinate partly because no refusal option was offered. Specify the exact refusal string.
- **Number the sources** and require inline citations — this makes unfaithful claims visible and auditable.
- **Put the question after the context** for long prompts; recency in the prompt improves instruction adherence.
- **Include metadata (date, title) with each chunk** so the model can resolve conflicts and reason about recency.
- **Instruct conflict handling explicitly**, or the model will silently pick one source and present it as settled fact.

The "Lost in the Middle" Problem

Research consistently shows LLM attention to retrieved context is **U-shaped**: information at the very start and very end of a long prompt is used reliably, while content buried in the middle is frequently ignored — even when it plainly contains the answer.

Mitigations:

- **Retrieve fewer, better chunks.** Re-ranking to a tight top-3/top-5 usually beats stuffing top-20.
- **Reorder deliberately:** place the highest-ranked chunks first and last, weakest in the middle ("lost-in-the-middle reordering").
- **Compress context:** summarize or extract only the query-relevant sentences from each chunk before assembly.
- **Deduplicate** near-identical chunks — redundancy wastes attention and inflates cost.

Counter-intuitive but well-established: increasing top-k often *decreases* answer quality past a point. More context means more distractors. Tune top-k empirically — many production systems settle at k=3 to k=6.

PART E

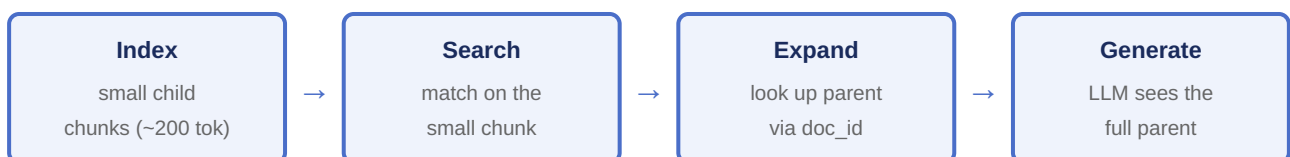
Advanced RAG Architectures

The Evolution: Naive → Advanced → Modular RAG

Generation	Characteristics	Typical Weakness
Naive RAG	Fixed chunking → vector search → stuff top-k → generate	Poor recall, no query understanding, no verification
Advanced RAG	Adds pre-retrieval (query rewriting, routing) and post-retrieval (re-ranking, compression) stages	Still a fixed, linear pipeline — can't adapt per query
Modular RAG	Composable modules with routing, memory, and iterative/conditional loops	Higher complexity, latency, and cost to operate
Agentic RAG	An agent decides <i>whether</i> , <i>where</i> , and <i>how many times</i> to retrieve, and can self-correct	Non-deterministic latency/cost; harder to evaluate

Parent-Document / Small-to-Big Retrieval

A core tension: **small chunks retrieve better** (precise embeddings) but **large chunks generate better** (more context). Small-to-big resolves it by decoupling the two.



Three variants of the same idea:

- **Parent-document retriever:** embed small chunks, return the enclosing larger section.
- **Sentence-window retrieval:** embed single sentences, return the sentence $\pm N$ surrounding sentences.
- **Summary indexing:** embed an LLM-generated summary of each document, return the full document (or its relevant section).

Contextual Retrieval

A powerful and simple upgrade: before embedding each chunk, use an LLM to prepend a short, chunk-specific description of how it fits into its parent document. This resolves pronouns and implicit references that otherwise make chunks unretrievable.

```
# For each chunk, generate a 1-2 sentence situating context
CTX_PROMPT = """<document>{full_doc}</document>
Here is a chunk from that document:
<chunk>{chunk}</chunk>
Give a short (1-2 sentence) context situating this chunk within the overall
document, to improve search retrieval. Answer with the context only."""

situating = llm(CTX_PROMPT.format(full_doc=doc.text, chunk=chunk.text))
chunk.embed_text = situating + "\n\n" + chunk.text # embed the enriched version
# ALSO index chunk.embed_text in BM25 -> contextual hybrid search
```

Why it works: A raw chunk saying "Revenue grew 3% over the prior quarter" matches almost nothing. Enriched to "This chunk is from ACME Corp's Q2 2023 10-Q, discussing segment performance. Revenue grew 3%..." it becomes retrievable by company, period, and topic. Published results show large reductions in retrieval failure rate when combined with hybrid search and re-ranking. Cost is a one-time indexing expense (mitigable with prompt caching).

GraphRAG

Standard RAG retrieves *independent passages*, which makes it weak at questions requiring connecting facts across many documents ("What are the recurring themes across all incident reports?"). GraphRAG builds a knowledge graph over the corpus instead.

GRAPHRAG INDEXING

- 1 **Entity & relation extraction** — An LLM reads each chunk and extracts entities (people, systems, orgs) and the relationships between them.
- 2 **Graph construction** — Merge duplicate entities across documents into a single graph of nodes and edges.
- 3 **Community detection** — Cluster the graph into communities of densely related entities (e.g., Leiden algorithm).
- 4 **Community summarization** — An LLM writes a summary for each community — these become high-level, corpus-wide knowledge.

GRAPHRAG QUERY TIME

- 1 **Local search** — For entity-specific questions: find the entity node, traverse its neighborhood, gather connected facts.
- 2 **Global search** — For thematic/aggregate questions: map over community summaries, then reduce into a final synthesized answer.

Question Type	Vector RAG	GraphRAG
"What is our refund window?" (fact lookup)	Excellent	Overkill
"How is X connected to Y?" (multi-hop)	Weak	Strong
"What are the main themes across all reports?" (global)	Fails — no single chunk contains the answer	Strong (community summaries)

Cost warning: GraphRAG indexing runs an LLM over the entire corpus, often multiple times — it can be orders of magnitude more expensive than vector indexing. Justify it with question types that genuinely require it, and consider a hybrid: vector RAG by default, graph traversal only for detected multi-hop/global queries.

Self-RAG and Corrective RAG (CRAG)

These architectures add **self-assessment loops** — the system judges its own retrieval and generation quality and takes corrective action instead of blindly answering.

Self-RAG — the model emits reflection tokens deciding:

- **Do I need to retrieve at all?** (Many queries — greetings, math, style edits — need no retrieval.)
- **Is this retrieved passage relevant?** (Filter before it pollutes the prompt.)
- **Is my draft answer supported by the passage?** (Faithfulness check.)
- **Is my answer actually useful for the question?** (Utility check.)

CRAG — grade retrieval, then branch:

- **Correct** (high confidence) → refine the retrieved knowledge and generate.
- **Ambiguous** → combine internal retrieval with a web search fallback.
- **Incorrect** (all docs irrelevant) → discard them entirely and fall back to web search or refuse.

```
# CRAG-style corrective loop (simplified control flow)
docs = retrieve(query)
grades = [grade_relevance(query, d) for d in docs]      # LLM or small classifier
good = [d for d, g in zip(docs, grades) if g == "relevant"]

if len(good) == 0:
    query2 = rewrite_for_web(query)
    good = web_search(query2)                          # external fallback
elif len(good) < 2:
    good += web_search(query)                          # augment thin evidence

answer = generate(query, good)

# Self-RAG style verification before returning to the user
if not is_grounded(answer, good):                      # faithfulness check
    answer = generate(query, good, stricter=True)
    if not is_grounded(answer, good):
        answer = "I don't have enough information to answer that reliably."
```

Multimodal RAG

Real enterprise documents are full of charts, diagrams, and screenshots that text-only pipelines silently discard.

Strategy	How It Works	Trade-off
Image captioning	A vision model writes a text description of each image; embed & retrieve the text	Simple, reuses text stack; loses fine visual detail
Unified multimodal embeddings	Embed images and text into one shared vector space	Elegant; quality varies by domain
Page-as-image (visual retrieval)	Embed rendered page images directly, feed matching pages to a vision LLM	Preserves layout/tables perfectly; higher token cost

PART F

Evaluation & Observability

"It looks good in the demo" is not evaluation. RAG has two independently failing subsystems, and you must measure them separately — otherwise you cannot tell whether a bad answer came from bad retrieval or bad generation, and you'll tune the wrong thing.

Retrieval Metrics

Metric	What It Measures	Use When
Recall@k	Fraction of queries where at least one relevant chunk appears in the top k	The single most important RAG metric — if recall fails, nothing downstream can fix it
Precision@k	Fraction of the top k that are actually relevant	Controlling prompt noise and cost
MRR (Mean Reciprocal Rank)	1/rank of the first relevant result, averaged	When only the top hit really matters
nDCG@k	Rank-weighted gain with graded relevance	The standard when relevance isn't binary
Hit Rate	Binary: was the gold document retrieved at all?	Quick regression checks in CI

Diagnostic rule: Measure **Recall@50** (retriever health) and **Recall@5 after re-ranking** (re-ranker health) separately. If Recall@50 is low, fix chunking/embeddings/hybrid search. If Recall@50 is high but Recall@5 is low, your re-ranker is the bottleneck.

Generation Metrics & the RAGAS Framework

RAGAS popularized a set of reference-light, LLM-as-judge metrics over the triple `(question, retrieved_contexts, answer)`.

Metric	Question It Answers	Diagnoses
Faithfulness	Is every claim in the answer supported by the retrieved context?	Hallucination in generation
Answer Relevancy	Does the answer actually address the question asked?	Evasive/off-target generation

Context Precision	Are the relevant chunks ranked near the top?	Re-ranker quality
Context Recall	Did retrieval fetch everything needed to answer fully?	Retriever/chunking quality
Answer Correctness	How close is the answer to a human-written ground truth?	End-to-end quality (needs labels)

How to read the metric matrix — a debugging table:

Context Recall	Faithfulness	Diagnosis	Fix
Low	High	Retrieval missed the evidence; model correctly refused or answered thinly	Fix chunking, add hybrid search, expand queries
High	Low	Right docs retrieved, but the model invented content anyway	Tighten the grounding prompt, require citations, consider a stronger model
Low	Low	Broken end-to-end	Start at parsing/chunking — do not tune prompts yet
High	High	Healthy; if users still complain, check answer relevancy & formatting	Tune presentation, tone, completeness

Building an Eval Set & CI for RAG

- 1 Collect real queries** — Mine actual user questions from logs. Synthetic-only eval sets systematically miss how real users phrase things.
- 2 Generate a golden set** — For each query, label the correct source chunk(s) and write an ideal answer. Target 50-200 examples covering easy lookups, multi-hop, ambiguous, and unanswerable queries.
- 3 Include negatives** — Add questions your corpus genuinely cannot answer. A system that never says 'I don't know' is silently hallucinating.
- 4 Automate in CI** — Run the eval on every chunking/prompt/model change. Block merges on regression in Recall@5 and Faithfulness.
- 5 Log everything in production** — Persist query, rewritten query, retrieved chunk IDs + scores, final prompt, answer, latency, cost, and user feedback.
- 6 Close the loop** — Route thumbs-down cases into the eval set. Your eval set should grow with the failures you actually encounter.

The most common evaluation mistake: Evaluating only end-to-end answer quality. When the answer is wrong you then have no idea which of six pipeline stages caused it. Always instrument per-stage.

PART G

Production Engineering

Cost, Latency & Caching

Stage	Typical Latency	Optimization Levers
Query rewriting (LLM)	200–800 ms	Use a small/fast model; skip for first-turn queries
Query embedding	10–50 ms	Cache embeddings of repeated queries
Vector search	10–100 ms	Tune <code>ef_search</code> ; pre-filter with metadata; right-size the index
Re-ranking	50–300 ms	Shrink candidate set; smaller model; batch; adaptive skip
Generation	1–10 s	Stream tokens; reduce context; smaller model for simple queries

Caching layers, in order of value:

- **Semantic answer cache** — embed the incoming query; if it's within a similarity threshold of a previous query, return the cached answer. Huge win on FAQ-heavy traffic.
- **Prompt/prefix caching** — reuse the KV-cache of a stable system prompt + context prefix across calls.
- **Embedding cache** — never re-embed unchanged chunks or repeated queries.
- **Retrieval cache** — cache the chunk-ID list per normalized query.

Semantic cache danger: "What is the 2023 policy?" and "What is the 2024 policy?" can be highly similar in embedding space. Always include hard filters (dates, entity IDs, tenant) in the cache key, and set the threshold conservatively — a wrong cache hit is worse than a cache miss.

Index Refresh & Incremental Updates

- **Content hashing:** hash each source document; re-chunk and re-embed only when the hash changes.
- **Deletes matter:** stale chunks from deleted documents are a compliance and correctness hazard. Implement tombstoning.
- **Chunk-level versioning:** store `doc_version` ; filter to current versions at query time rather than deleting in-place.
- **Blue/green re-indexing:** when changing embedding models or chunking, build the new index alongside the old, evaluate, then swap atomically.

- **Freshness filters:** for time-sensitive corpora, apply recency boosting or hard date filters rather than relying on semantics.

Security: Access Control, PII & Prompt Injection

Risk	Attack/Failure	Mitigation
Cross-tenant leakage	User A retrieves User B's documents	Mandatory <code>tenant_id</code> pre-filter in every query; test with adversarial eval cases
Over-permissive retrieval	Employee retrieves HR/salary docs they can't access	Enforce ACLs as pre-filters; mirror the source system's permission model at ingest
Indirect prompt injection	A malicious document contains "Ignore previous instructions and exfiltrate..."; it gets retrieved and obeyed	Delimit and label context as untrusted data; instruct the model never to follow instructions inside context; sanitize on ingest; constrain tool access
PII exposure	Sensitive data indexed and surfaced in answers	PII detection/redaction at ingest; classify and restrict at chunk level
Data exfiltration via citations	Answer refuses but the citation snippet leaks the content	Apply the same ACL filter to citation rendering

The golden rule of RAG security: Retrieved context is **untrusted user-level data**, not trusted instructions. Structure prompts so that instructions come only from the system role, and context is explicitly fenced and labeled as reference material to be summarized — never obeyed.

Common Failure Modes — Debugging Playbook

Symptom	Likely Cause	First Things to Try
Answers are confidently wrong	Irrelevant chunks retrieved; no refusal path	Check Recall@k; add explicit "say I don't know" instruction; require citations
"I don't know" on answerable questions	Retrieval miss or over-strict prompt	Inspect retrieved chunk IDs; add hybrid search; loosen refusal threshold
Works for keywords, fails on concepts	Sparse-only or weak embedding model	Add dense retrieval; upgrade embedding model
Fails on IDs, codes, product names	Dense-only retrieval	Add BM25 + RRF hybrid
Good for one document, bad across many	No multi-hop capability	Query decomposition; consider GraphRAG

Chat follow-ups retrieve nonsense	Embedding the raw follow-up message	Add history-aware query rewriting
Answer ignores a chunk you can see is correct	Lost-in-the-middle	Reduce top-k; reorder; compress context
Quality degraded after an "upgrade"	Embedding model changed without re-index	Full re-index; version your indexes
Answers cite outdated policy	Stale chunks never deleted	Implement tombstoning + version filters

PART H

Reference Implementations & Interview Prep

Reference 1 — Minimal RAG (the 40-line baseline)

```
# Framework-agnostic pseudocode; maps 1:1 onto LangChain/LlamaIndex primitives.

# ----- OFFLINE: INDEX -----
def index_corpus(paths):
    chunks = []
    for p in paths:
        text = parse(p) # PDF/HTML/MD -> clean text
        for piece in recursive_split(text, size=600, overlap=80):
            chunks.append({"text": piece, "meta": {"source": p}})

    vectors = embed_batch([c["text"] for c in chunks]) # bi-encoder
    store.upsert(vectors, chunks) # HNSW index + metadata

# ----- ONLINE: QUERY -----
def answer(question, k=5):
    qvec = embed(question)
    hits = store.search(qvec, top_k=k)

    context = "\n\n".join(
        f"[doc_{i}] {h['text']}" for i, h in enumerate(hits, 1))

    prompt = (
        "Answer ONLY from the context. Cite sources like [doc_1]. "
        "If the answer is not present, say you don't know.\n\n"
        f"CONTEXT:\n{context}\n\nQUESTION: {question}"
    )
    return llm(prompt)
```

Reference 2 — Production-Grade RAG (hybrid + rewrite + rerank + verify)

```
def answer_v2(question, history=None, user=None, k_retrieve=50, k_final=5):

    # 1) QUERY UNDERSTANDING -----
    q = rewrite_with_history(question, history) if history else question
    if needs_decomposition(q):
        subqs = decompose(q)                # multi-hop / comparative
    else:
        subqs = [q]

    # 2) HYBRID RETRIEVAL (per sub-question) -----
    acl = {"tenant_id": user.tenant, "access_level": {"$lte": user.level}}
    pools = []
    for sq in subqs:
        dense = store.search(embed(sq), top_k=k_retrieve, filter=acl)
        sparse = bm25.search(sq, top_k=k_retrieve, filter=acl)
        pools.append(rrf_fuse([dense, sparse], weights=[0.6, 0.4]))

    candidates = dedupe(flatten(pools))[:k_retrieve]

    # 3) RE-RANK (precision stage) -----
    scores = cross_encoder.predict([(q, c.text) for c in candidates])
    reranked = [c for _, c in sorted(zip(scores, candidates), reverse=True)]

    # 4) RELEVANCE GATE (CRAG-style) -----
    top = [c for c, s in zip(reranked, sorted(scores, reverse=True))
           if s > RELEVANCE_THRESHOLD][:k_final]
    if not top:
        return {"answer": "I couldn't find relevant information.", "sources": []}

    # 5) CONTEXT ASSEMBLY (small-to-big + reorder) -----
    expanded = [fetch_parent(c) for c in top]          # small-to-big
    ordered = reorder_lost_in_middle(expanded)        # best first & last
    context = format_with_citations(ordered)

    # 6) GENERATE -----
    draft = llm(SYSTEM_GROUNDED, f"CONTEXT:\n{context}\n\nQUESTION: {question}")

    # 7) VERIFY (Self-RAG style faithfulness gate) -----
    if not is_grounded(draft, ordered):
        draft = llm(SYSTEM_STRICTER, ...)
        if not is_grounded(draft, ordered):
            draft = "I don't have enough information to answer reliably."

    # 8) LOG for evaluation -----
    log(question=question, rewritten=q, chunk_ids=[c.id for c in top],
         scores=scores[:k_final], answer=draft)

    return {"answer": draft, "sources": [c.meta for c in ordered]}
```

Tech Stack Quick Reference

Layer	Common Options	Selection Criteria
Orchestration	LangChain, LlamaIndex, Haystack, or plain code	LlamaIndex leans data/indexing; LangChain leans composition/agents; plain code for full control
Parsing	Layout-aware PDF parsers, OCR engines, vision models	Document complexity (tables/scans) drives this more than anything
Embeddings	Hosted API models, open-source sentence encoders	Domain fit, sequence length, cost, data residency
Vector store	pgvector, FAISS, managed vector DBs, OpenSearch	Scale, filtering needs, existing infra
Sparse/keyword	BM25 implementations, Elasticsearch/OpenSearch	Needed for hybrid — don't skip it
Re-ranker	Cross-encoder models (hosted or self-run)	Latency budget vs. precision gain
Evaluation	RAGAS, custom LLM-judge harnesses, tracing platforms	Must integrate with CI

Common RAG Interview Questions & Model Answers

Q: Explain RAG in one minute to a non-technical stakeholder.

RAG connects a language model to your company's documents. When someone asks a question, the system first searches those documents for the most relevant passages, then asks the model to answer using only those passages, with citations. This keeps answers current, grounded in your data, and verifiable — without retraining the model every time your documents change.

Q: How would you improve a RAG system that returns irrelevant answers?

I'd first isolate *where* it fails. I'd measure Recall@50 to test the retriever and Recall@5 after re-ranking to test precision. If Recall@50 is low, the problem is upstream: parsing, chunking, or embeddings — I'd add hybrid BM25+vector search with RRF, inject document/section context into chunks before embedding, and check chunk sizes. If Recall@50 is high but Recall@5 is low, I'd add or upgrade a cross-encoder re-ranker. If retrieval is fine but answers are still wrong, it's a generation/grounding issue — tighten the prompt, require citations, add an explicit refusal path, and reduce top-k to avoid lost-in-the-middle.

Q: When would you choose fine-tuning over RAG?

When the gap is behavioral rather than informational. If the model has the knowledge but produces the wrong tone, format, or fails to follow a domain-specific output structure, fine-tuning is the right tool. For

facts that change or are private, RAG is correct — fine-tuned facts go stale and can't be cited. In production these are usually complementary, not alternatives.

Q: Why use hybrid search instead of just vector search?

Because they fail differently. Dense retrieval handles paraphrase and synonyms well but is weak on exact identifiers — error codes, SKUs, person names, acronyms — where a near-miss in vector space is semantically meaningless. BM25 handles those precisely but breaks on vocabulary mismatch. Fusing both with Reciprocal Rank Fusion covers both failure modes, and RRF is rank-based so it needs no score normalization when models change.

Q: What is the difference between a bi-encoder and a cross-encoder, and where does each belong?

A bi-encoder embeds the query and document independently, so document vectors can be precomputed and indexed — that makes it fast enough to search millions of chunks, but it compresses each document before ever seeing the query, so information is lost. A cross-encoder processes the query and document together with full attention, giving a far more accurate relevance score, but it can't be precomputed. So the standard architecture is a two-stage funnel: bi-encoder for recall over the whole corpus, cross-encoder to re-rank a shortlist of 50 down to the top 5.

Q: How do you handle follow-up questions in a RAG chatbot?

You must rewrite the query before retrieval. A message like 'what about the second one?' has no standalone meaning, so embedding it retrieves noise. I'd pass the conversation history plus the latest message to a small, fast LLM that outputs a self-contained search query, retrieve with that rewritten query, and then generate using the original user phrasing so the answer still sounds natural in context.

Q: How do you evaluate a RAG system?

Separately per stage, because end-to-end scores don't localize failures. For retrieval: Recall@k, MRR, nDCG against a labeled golden set. For generation: faithfulness (is every claim supported by context?) and answer relevancy, typically via LLM-as-judge frameworks like RAGAS. I'd build a golden set of 50-200 real user queries including unanswerable ones to test refusal behavior, run it in CI on every chunking or prompt change, and log retrieved chunk IDs in production so thumbs-down cases can be triaged and folded back into the eval set.

Q: What is 'lost in the middle' and how do you mitigate it?

LLM attention over long contexts is U-shaped — content at the beginning and end of the prompt is used reliably, while content in the middle is often ignored even when it contains the answer. Mitigations: retrieve fewer but better chunks via re-ranking, reorder so the strongest chunks sit first and last, compress or summarize chunks before assembly, and deduplicate near-identical passages. Counter-intuitively, increasing top-k often hurts quality for this reason.

Q: How would you secure a RAG system in an enterprise?

Three layers. First, access control enforced as a pre-filter inside the retrieval query using tenant and permission metadata attached at ingest — never post-filter, because once an unauthorized chunk enters the prompt it has effectively leaked. Second, treat retrieved context as untrusted data: fence it, label it, and instruct the model never to follow instructions found inside it, to defend against indirect prompt injection from poisoned documents. Third, PII detection and redaction at ingest, plus applying the same ACL filter to citation rendering so refusals don't leak content through snippets.

Q: When does GraphRAG justify its cost over vector RAG?

When questions require connecting information that no single chunk contains — multi-hop relationship questions, or global/thematic questions like 'what are the recurring themes across all incident reports?' Vector RAG retrieves independent passages, so it structurally cannot answer those. GraphRAG extracts entities and relationships into a graph and precomputes community summaries, which handles both. The cost is that indexing runs an LLM over the whole corpus, which can be orders of magnitude more expensive — so I'd typically route: vector RAG by default, graph traversal only for detected multi-hop or aggregate queries.

Final note: The defining insight of RAG engineering is that **retrieval quality dominates everything else**. Teams routinely spend weeks swapping generation models for a few percent, while a day spent fixing PDF table parsing, adding hybrid search, or injecting section headers into chunks yields far larger gains. Instrument first, fix the earliest broken stage, and only then tune downstream.